TECHNICAL SPECIFICATION

Delivery and Order Tracking System

****Project Type:**** Backend REST API

****Developer:**** Adeife Ajala

****Purpose:**** Women Techsters Mentorship Programme

****Status:**** Draft for Review

---

1. Project Overview

1.1 Introduction

The Delivery and Order Tracking System is a backend application designed to manage delivery requests, order processing, delivery assignments, payments, notifications, and real-time delivery tracking.

The system will connect customers who request deliveries with delivery riders responsible for transporting packages from pickup locations to designated destinations.

It will provide administrators with tools to manage orders, assign riders, monitor delivery progress, and oversee payment transactions.

The application will expose a RESTful API for client applications and use real-time communication to deliver timely updates on order status and rider location.

1.2 Problem Statement

Managing deliveries manually can create several operational challenges, including:

- Difficulty tracking orders from creation to completion.
- Delays in assigning deliveries to available riders.
- Poor visibility into delivery progress for customers.
- Inconsistent handling of payment records.
- Delayed communication when order statuses change.
- Limited accountability when deliveries fail or encounter problems.

The system aims to address these challenges by centralising delivery operations, enforcing clear order workflows, and providing reliable tracking and notification mechanisms.

1.3 Project Objectives

The objectives are to:

1. Build a secure backend API for managing customers, orders, and deliveries.
2. Implement a structured delivery assignment process.
3. Maintain a reliable order and delivery status lifecycle.
4. Integrate payment processing and transaction verification.

5. Provide real-time delivery status and location updates.

6. Implement notifications for important order events.

7. Apply appropriate security, validation, and access-control mechanisms.

8. Test the system and deploy it using an automated development workflow.

---

2. Project Scope

2.1 Core Features

The first version will include the following modules:

****1. Authentication and User Management****

• Customer registration and login.

• Secure password hashing.

• JWT access and refresh tokens.

• Role-based access control.

• User profile management.

****2. Order Management****

• Create delivery orders.

• View order details and order history.

• Specify pickup and delivery addresses.

• Record package descriptions and delivery instructions.

• Calculate or record delivery fees.

• Cancel eligible orders.

• Track order status.

****3. Delivery Assignment****

• Allow administrators to assign riders to orders.

• Allow riders to view their assigned deliveries.

• Allow riders to accept or reject assignments.

• Prevent conflicting active assignments.

• Record assignment history.

****4. Delivery Tracking****

• Provide customers with delivery status updates.

• Allow riders to submit location updates while delivering an order.

• Deliver real-time updates through WebSockets.

• Store relevant delivery status history.

• Restrict access to tracking information to authorised users.

****5. Payment Management****

• Initiate payments for delivery orders.

• Record payment attempts and transaction references.

• Verify payments with the payment provider.

• Process payment webhooks securely.

• Track payment status.

• Handle duplicate payment notifications safely.

****6. Notifications****

• Notify customers when orders are created.

• Notify customers when a rider is assigned.

• Send updates when delivery status changes.

• Notify customers when deliveries are completed or fail.

• Notify riders when new assignments are made.

****7. Administration****

• View and manage orders.

• Manage customer and rider accounts.

• Assign and reassign riders where permitted.

• Monitor delivery progress.

• View payment records.

• Review order and delivery histories.

2.2 Out of Scope for the Initial Version

To keep the project manageable, the first release will not include:

• Multiple delivery companies operating on the same platform.

• Advanced route optimisation.

• Machine-learning-based delivery predictions.

• Complex rider earnings and commission calculations.

• A full customer or rider mobile application.

• Multi-currency payment processing.

These features can be considered after the core system is working reliably.

---

3. User Roles and Permissions

The system will initially support three user roles.

3.1 Customer

Customers will be able to:

• Register and authenticate.

• Manage their profiles.

• Create delivery orders.

• View their own orders.

• Make payments.

• Track their deliveries.

• Receive notifications.

• Cancel orders when cancellation rules permit.

• View their payment and order histories.

Customers must not be able to access another customer's private order information.

3.2 Delivery Rider

Riders will be able to:

• Authenticate.

• View available assignments or assignments made to them.

• Accept or reject eligible assignments.

• View pickup and delivery details for authorised assignments.

• Update delivery status.

• Submit location updates during active deliveries.

• View their delivery history.

Riders must not be able to access unrelated customers' orders or modify payment records.

3.3 Administrator

Administrators will be able to:

• View and manage orders.

• Manage customer and rider accounts.

• Assign riders to orders.

• Reassign deliveries when necessary.

• Monitor delivery statuses.

• View payment records and delivery histories.

• Investigate failed deliveries and operational issues.

Administrative actions should be authenticated, authorised, validated, and logged where appropriate.

---

4. Proposed Technology Stack

| Component | Technology | Purpose |

| ---------- | ---------- | ---------- |

| Runtime | Node.js | Server-side execution |

| Language | TypeScript | Type safety and maintainability |

| Backend framework | NestJS | Modular application architecture |

| Database | PostgreSQL | Persistent relational data |

| ORM | Prisma | Database access and migrations |

| Authentication | Passport.js and JWT | Authentication and access control |

| Real-time communication | Socket.IO | Delivery status and location updates |

| Caching and coordination | Redis | Caching and selected background or coordination tasks |

| Background jobs | BullMQ | Asynchronous notifications and other queued tasks |

| API documentation | Swagger / OpenAPI | Interactive API documentation |

| Validation | class-validator and class-transformer | Request validation and transformation |

| Testing | Jest and Supertest | Unit and API integration testing |

| Containerisation | Docker and Docker Compose | Consistent development and deployment environments |

| CI/CD | GitHub Actions | Automated checks and build workflows |

| Deployment | Render or another suitable hosting platform | Hosting the API and database |

| Logging | Structured application logging | Troubleshooting and operational visibility |

****Architecture decision:**** Begin with a modular monolith rather than microservices. This keeps development manageable while allowing the application to be divided into clearly defined modules.

---

5. System Architecture

The system will follow a modular backend architecture.

5.1 High-Level Request Flow

A client sends a request to the API. NestJS authenticates the user, checks permissions, validates the request, and passes the operation to the relevant service.

The service applies the business rules and uses Prisma to interact with PostgreSQL.

Where required, the application queues background jobs, sends notifications, and publishes real-time events through Socket.IO.

5.2 Proposed Modules

• ****Auth Module:**** Registration, login, token refresh, and authentication.

• ****Users Module:**** User profiles and account management.

• ****Orders Module:**** Order creation, retrieval, cancellation, and lifecycle management.

• ****Deliveries Module:**** Rider assignments and delivery operations.

• ****Tracking Module:**** Location updates and real-time tracking.

• ****Payments Module:**** Payment initiation, verification, and webhook processing.

• ****Notifications Module:**** Email and other supported notifications.

• ****Admin Module:**** Administrative operations.

• ****Health Module:**** Health checks and application readiness.

Each module should have a clear responsibility and expose only the functionality needed by other modules.

---

6. Core Business Workflows

6.1 Order Creation

1. A customer authenticates.

2. The customer submits an order containing pickup details, delivery details, and package information.

3. The API validates the request.

4. The system calculates or records the delivery fee according to the configured pricing rules.

5. The system creates the order with an initial status.

6. The customer receives an order reference and confirmation.

7. The system makes the order available for the next appropriate operational step.

6.2 Payment Processing

1. The customer initiates payment for an eligible order.

2. The API creates a payment record with a unique transaction reference.

3. The payment provider processes the payment.

4. The backend verifies the transaction with the provider or processes an authenticated webhook.

5. The system updates the payment status after successful verification.

6. The order becomes eligible for the next step according to the business rules.

A payment must not be marked successful merely because a client claims that payment succeeded.

Payment callbacks must be verified, and duplicate webhook events must not cause duplicate payment records or repeated business actions.

6.3 Rider Assignment

1. An eligible order becomes available for assignment.

2. An administrator selects an available rider.

3. The system validates the rider and order.

4. The assignment is recorded.

5. The rider receives a notification.

6. The rider accepts or rejects the assignment.

7. The system updates the assignment and order workflow accordingly.

The system must prevent an order from having conflicting active assignments.

6.4 Delivery Tracking

1. A rider accepts an assignment.

2. The rider begins the delivery.

3. The rider submits location updates while the delivery is active.

4. The backend validates the rider's authorisation and the update.

5. The system records the latest location and publishes appropriate tracking events.

6. Authorised customers receive updates through a WebSocket connection.

7. The rider updates the delivery status at important milestones.

8. The system records the status history and notifies relevant users.

Location tracking should stop when the delivery is completed or cancelled, according to the system's rules.

6.5 Delivery Completion

1. The rider marks the delivery as completed.

2. The API validates that the current state permits completion.

3. The system records the completion timestamp.

4. The order and delivery statuses are updated consistently.

5. A completion notification is queued.

6. The customer can view the completed delivery in their order history.

---

7. Order and Delivery Statuses

Order and delivery statuses should be managed separately because they represent different aspects of the workflow.

7.1 Order Status

Proposed order statuses:

- `PENDING_PAYMENT`
- `CONFIRMED`
- `CANCELLED`

Additional statuses may be introduced if the final workflow requires them.

7.2 Delivery Status

Proposed delivery statuses:

- `PENDING_ASSIGNMENT`
- `ASSIGNED`
- `ACCEPTED`
- `PICKED_UP`
- `IN_TRANSIT`
- `DELIVERED`
- `FAILED`
- `CANCELLED`

The API must enforce valid transitions. For example, a rider must not mark an unassigned delivery as picked up.

Payment status must be tracked separately from order and delivery status.

---

8. Database Design

The initial relational data model will contain the following entities.

8.1 User

Stores account and authentication information.

Proposed fields:

- `id`
- `fullName`
- `email`
- `passwordHash`
- `phoneNumber`
- `role`
- `isActive`
- `createdAt`

• `updatedAt`

8.2 Order

Stores customer order information.

Proposed fields:

• `id`
• `orderNumber`
• `customerId`
• `pickupAddress`
• `pickupLatitude`
• `pickupLongitude`
• `deliveryAddress`
• `deliveryLatitude`
• `deliveryLongitude`
• `packageDescription`
• `deliveryInstructions`
• `deliveryFee`
• `currency`
• `status`
• `createdAt`
• `updatedAt`

8.3 Delivery

Stores delivery execution information.

Proposed fields:

- `id`
- `orderId`
- `riderId`
- `status`
- `assignedAt`
- `acceptedAt`
- `pickedUpAt`
- `deliveredAt`
- `failureReason`
- `createdAt`
- `updatedAt`

8.4 Payment

Stores payment transaction information.

Proposed fields:

- `id`
- `orderId`
- `providerReference`
- `amount`
- `currency`
- `status`
- `paidAt`
- `createdAt`
- `updatedAt`

8.5 DeliveryLocation

Stores relevant rider location updates.

Proposed fields:

- `id`
- `deliveryId`
- `latitude`
- `longitude`
- `recordedAt`

Location retention and update frequency will need to be defined to avoid storing unnecessary location data.

8.6 Notification

Stores notification records.

Proposed fields:

- `id`
- `userId`
- `type`
- `title`
- `message`
- `status`
- `sentAt`
- `createdAt`

8.7 DeliveryStatusHistory

Records changes to delivery status.

Proposed fields:

- `id`
- `deliveryId`

• `previousStatus`

• `newStatus`

• `changedBy`

• `createdAt`

The final Prisma schema will define relationships, indexes, unique constraints, and deletion behaviour.

---

9. API Design

The API will use versioned REST endpoints under `/api/v1`.

9.1 Authentication

| Method | Endpoint | Purpose |

| ------ | ---------------------- | -- |

| POST | `/api/v1/auth/register` | Register an account |

| POST | `/api/v1/auth/login` | Authenticate a user |

| POST | `/api/v1/auth/refresh` | Refresh an access token |

| POST | `/api/v1/auth/logout` | End the authenticated session according to the token strategy |

| GET | `/api/v1/users/me` | Retrieve the authenticated user's profile |

| PATCH | `/api/v1/users/me` | Update the authenticated user's profile |

9.2 Orders

| Method | Endpoint | Purpose |

| ------ | ---------- | ---------- |

| POST | `/api/v1/orders` | Create an order |

| GET | `/api/v1/orders` | Retrieve orders visible to the authenticated user |

| GET | `/api/v1/orders/:id` | Retrieve an order by ID |

| POST | `/api/v1/orders/:id/cancel` | Cancel an eligible order |

| GET | `/api/v1/orders/:id/history` | Retrieve relevant order or delivery history |

9.3 Deliveries

| Method | Endpoint | Purpose |

| ------ | ---------- | ---------- |

| GET | `/api/v1/deliveries/assigned` | Retrieve the authenticated rider's assignments |

| GET | `/api/v1/deliveries/:id` | Retrieve an authorised delivery |

| POST | `/api/v1/deliveries/:id/accept` | Accept a delivery assignment |

| POST | `/api/v1/deliveries/:id/reject` | Reject an eligible assignment |

| POST | `/api/v1/deliveries/:id/pickup` | Record package pickup |

| POST | `/api/v1/deliveries/:id/start` | Start the delivery |

| POST | `/api/v1/deliveries/:id/complete` | Mark the delivery as completed |

| POST | `/api/v1/deliveries/:id/fail` | Record a failed delivery |

9.4 Tracking

| Method | Endpoint | Purpose |

| ------ | ---------- | ---------- |

| POST | `/api/v1/deliveries/:id/location` | Submit a rider location update |

| GET | `/api/v1/deliveries/:id/tracking` | Retrieve authorised tracking information |

WebSocket events will support live tracking and delivery status notifications.

9.5 Payments

| Method | Endpoint | Purpose |

| ------ | ---------- | ---------- |

| POST | `/api/v1/orders/:id/payments` | Initiate payment |

| GET | `/api/v1/payments/:id` | Retrieve an authorised payment record |

| POST | `/api/v1/payments/verify` | Verify a payment transaction |

| POST | `/api/v1/payments/webhook` | Receive payment-provider events |

The payment endpoints will be adjusted to the chosen provider's integration requirements.

9.6 Administration

| Method | Endpoint | Purpose |

| ------ | ---------- | ---------- |

| GET | `/api/v1/admin/orders` | View and filter orders |

| POST | `/api/v1/admin/orders/:id/assign` | Assign a rider |

| POST | `/api/v1/admin/deliveries/:id/reassign` | Reassign an eligible delivery |

| GET | `/api/v1/admin/riders` | View and manage rider accounts |

| GET | `/api/v1/admin/payments` | View payment records |

All administrative endpoints must require the appropriate permissions.

---

10. Real-Time Communication

Socket.IO will provide real-time delivery tracking and status updates.

Proposed Events

****Client-to-server events:****

• `delivery:join`

• `delivery:leave`

Rider location submissions will use an authenticated API endpoint initially, with WebSocket-based location updates considered if needed.

****Server-to-client events:****

• `delivery:assigned`

• `delivery:status_updated`

• `delivery:location_updated`

• `delivery:completed`

• `delivery:failed`

The server must authenticate connections, verify delivery access, and ensure that customers cannot subscribe to unrelated deliveries.

The database remains the source of truth. A WebSocket event is a notification of a change, not a replacement for persisting the change.

---

11. Security Requirements

The system will implement the following security measures:

• Hash passwords using bcrypt or another suitable password-hashing algorithm.

• Protect private endpoints with authentication and role-based authorisation.

• Validate and transform incoming request data.

• Apply rate limiting to sensitive endpoints.

• Use Helmet and appropriate HTTP security settings.

• Keep credentials and secrets in environment variables.

• Validate payment webhooks using the payment provider's verification mechanism.

• Use unique transaction references and idempotent payment processing.

• Prevent users from accessing orders and deliveries they do not own.

• Validate state transitions to prevent unauthorised status changes.

• Restrict access to rider location information.

• Log relevant security and administrative events without exposing sensitive information.

---

12. Background Jobs and Notifications

BullMQ and Redis may be used for asynchronous work.

Proposed background tasks include:

• Sending order confirmation notifications.

• Notifying riders of assignments.

• Sending delivery status notifications.

• Retrying eligible failed notification attempts.

• Processing deferred tasks that do not need to block an API response.

Jobs should be designed to tolerate retries without sending duplicate notifications unnecessarily.

Critical order and payment changes must be persisted before dependent notifications are dispatched.

---

13. Error Handling and Observability

The API will use consistent error responses and structured logs.

Common HTTP Status Codes

- `200 OK`: Request completed successfully.
- `201 Created`: Resource created successfully.
- `400 Bad Request`: Invalid request or invalid state transition.
- `401 Unauthorized`: Authentication required or invalid.
- `403 Forbidden`: Insufficient permissions.
- `404 Not Found`: Resource does not exist or is not accessible.
- `409 Conflict`: Conflicting operation or resource state.
- `422 Unprocessable Entity`: Optional use for semantically invalid requests.
- `429 Too Many Requests`: Rate limit exceeded.
- `500 Internal Server Error`: Unexpected server failure.

The system should provide:

- A health-check endpoint.

• Structured application logging.

• Request correlation IDs.

• Appropriate database and dependency error reporting.

• Metrics for operational visibility where practical.

Sensitive information, passwords, access tokens, and payment secrets must not appear in logs.

---

14. Testing Strategy

Testing will cover the main business workflows and failure cases.

Unit Tests

• Order status transition rules.

• Delivery assignment rules.

• Payment state handling.

• Access-control decisions.

• Notification job behaviour.

Integration Tests

• Database persistence.

• Order creation and retrieval.

• Rider assignment.

• Payment record updates.

• Delivery status history.

• Transaction rollback behaviour.

End-to-End Tests

• Customer registration and login.

• Creating and cancelling eligible orders.

• Assigning and accepting deliveries.

• Preventing conflicting assignments.

• Validating payment verification.

• Restricting access to other users' orders.

• Completing deliveries and recording history.

• Testing important concurrent operations.

Real-Time Tests

• Authorised clients receive delivery updates.

• Unauthorised users cannot subscribe to private delivery information.

• Location updates are rejected for invalid delivery states.

---

15. Deployment and CI/CD

The project will use Docker to provide a consistent development environment.

GitHub Actions will automate relevant quality checks, including:

- Dependency installation.
- Linting and formatting checks where configured.
- TypeScript compilation and application builds.
- Unit tests.
- Integration and end-to-end tests against PostgreSQL.
- Docker image builds.

The application will be deployed to a suitable hosting platform with a managed or hosted PostgreSQL database.

Production configuration will be supplied through environment variables. Database migrations, health checks, logging, and secret management will be addressed before deployment.

Redis-dependent functionality will be configured and verified separately before being described as active in production.

---

16. Non-Functional Requirements

Reliability

The system should preserve consistent order, delivery, and payment records during expected failures.

Security

Users must only access information and perform actions permitted by their roles.

Maintainability

The codebase should use modular NestJS architecture, clear naming, and documented API contracts.

Performance

Database queries should be indexed appropriately, and expensive work should be moved to background jobs where suitable.

Scalability

The design should allow the API to be scaled without relying on process-local state for functionality that must be shared between instances.

Privacy

Customer information and delivery location data should only be collected, stored, and exposed when required for the delivery workflow.

---

17. Implementation Milestones

Phase 1: Planning and Foundation

- Finalise the technical specification.
- Design the database schema.
- Set up the NestJS project.
- Configure PostgreSQL, Prisma, Docker, and environment variables.
- Establish GitHub repository conventions and CI checks.

Phase 2: Authentication and Orders

- Implement registration and login.
- Add roles and permissions.
- Build order creation, retrieval, and cancellation.
- Add order status validation and tests.

Phase 3: Delivery Management

• Implement rider management and assignments.

• Build assignment acceptance and rejection.

• Implement delivery status transitions and history.

• Add tests for conflicting assignments.

Phase 4: Payments

• Select and integrate a payment provider or sandbox.

• Implement payment initiation and verification.

• Process payment webhooks safely.

• Test duplicate events and unsuccessful payments.

Phase 5: Tracking and Notifications

• Implement location updates.

• Add Socket.IO tracking events.

• Implement notification jobs and retries.

• Test authorisation and real-time event delivery.

Phase 6: Quality and Deployment

• Complete integration and end-to-end tests.

• Review security and error handling.

• Configure deployment and production environment variables.

• Verify health checks and database connectivity.

• Publish API documentation and update the README.

---

18. Definition of Done

The initial release will be considered complete when:

1. Customers can create and view their own delivery orders.

2. Administrators can assign riders to eligible orders.

3. Riders can manage assigned deliveries through valid status transitions.

4. The system prevents conflicting active assignments.

5. Payment records are verified and updated safely.

6. Authorised customers can receive delivery status and location updates.

7. Notifications are processed with appropriate failure handling.

8. Access controls and request validation are tested.

9. The automated test suite passes.

10. The API is deployed, documented, and accessible through a verified health endpoint.

---

19. Open Decisions for Mentor Review

Before implementation begins, the following decisions should be confirmed:

1. ****Delivery model:**** Will customers request delivery of their own packages, or will the system also support merchants creating orders for customers?

2. ****Rider assignment:**** Will administrators assign riders manually in the first version, or should the system automatically select riders?

3. ****Payment timing:**** Must customers pay before a rider is assigned, or can payment happen at another stage?

4. ****Payment provider:**** Which provider or sandbox should be used for payment integration?

5. ****Tracking:**** Is live rider location tracking required for the first release, or should the first version focus on real-time delivery status updates?

6. ****Notifications:**** Are email notifications sufficient for the first version, or are additional channels required?

7. ****Delivery pricing:**** Should the fee be fixed, manually configured, or calculated from distance?

8. ****Project priorities:**** Which features should be treated as mandatory for the mentorship deadline?

These decisions will determine the final requirements, database relationships, endpoint contracts, and implementation order.